Assignment 2: Implementation Reference

GStar Bootcamp

Abstract

Triton shapes, debugging invariants, every named student region, reusable primitives, and a checker cookbook.

Companions: Overview, Final KDA problem.

A Kernel interface, notation, and setup

Common tensor layout

The wrappers accept noncontiguous tensors and prepare contiguous GPU buffers in the following layout. Key and value dimensions may differ.

Table 1. Common tensor layout.

Tensor	Shape	Meaning
`q`, `k`	(B, H, T, D_k)	Query and key vectors
`v`, `out`	(B, H, T, D_v)	Value and output vectors
`initial_state`	(B, H, D_k, D_v)	Matrix memory before token 1
`final_state`	(B, H, D_k, D_v)	Matrix memory after token T
`beta`	(B, H, T)	Scalar delta write strength
Scalar `gate`	(B, H, T)	Natural log of a scalar forget gate
Channel `gate`	(B, H, T, D_k)	Natural log of one gate per key channel

The starter pack supports FP32, FP16, or BF16 queries, keys, and values with a shared dtype. Use FP32 gates and write strengths; $D_k, D_v \in \{64, 128\}$; chunk length $C = 64$; and secondary tile length $R = 16$. Write strengths satisfy $\beta \in [0, 1]$, including endpoint tests. Accumulated states, log-gate prefix sums, and reduction accumulators use FP32. The reference uses FP64 on small test cases.

Shared forward contract

With `output_final_state=True`, every completed attention function returns `(out, final_state)`. The default call returns only `out`. Missing initial state means zero. GPU final states are FP32; P1 keeps FP64 state for FP64 inputs. Output has the value input's dtype. The optional `scale` argument defaults to the inverse square root of the key dimension. Apply the scale once, during the output read. Inputs and initial state must not be modified.

Environment and first commands

Use Python 3.10 or newer and PyTorch. Problems 2–9 additionally need Linux, Triton, and an NVIDIA CUDA GPU. Problem 1 runs on CPU, including macOS. Dependency ranges in

`requirements.txt` are an initial setup specification; they have not yet been frozen against a course GPU.

```
cd GStar-Assignment-2
python -m pip install -r requirements.txt
python autograder.py --check-templates
python autograder.py --p1 --device cpu
python autograder.py --p3 --dtype float32
python autograder.py --p9 --dtype bfloat16
```

The static audit checks syntax, public entry points, dependency direction, and forbidden reference imports. It does not compile a GPU kernel or check numerical correctness. The numerical commands deliberately fail until you complete their required student regions.

Understand the call contract before writing a kernel

```
# q/k: [B, H, T, Dk]; v: [B, H, T, Dv]
# s0: [B, H, Dk, Dv], floating-point matrix memory
out, final_state = attention(
    q, k, v, ..., scale=0.37,
    initial_state=s0, output_final_state=True,
)
# With output_final_state=False (the default), return out alone.
```

Extra inputs are: none for linear attention; a channel gate for GLA; `beta` for DeltaNet; scalar gate and `beta` for Gated DeltaNet; channel gate and `beta` for KDA. Query scale affects outputs only. Zero scale yields zero output with unchanged state evolution.

Names in the starter code

Python launchers use `batch_size`, `num_heads`, `head_dim` and `value_dim` for tensor dimensions. In a public chunk launcher, `seq_len` is the original token count, `padded_len` is the count after neutral padding, and `num_chunks=padded_len//CHUNK_SIZE`. Internal helpers receive padded inputs and call their token count `seq_len`. These names follow the reference kernel conventions; all executable dependencies remain inside this folder.

Uppercase Triton arguments such as `Q`, `K` and `States` are pointers. Lowercase `q`, `k` and `state` are loaded tiles. Blank lines separate loads, student arithmetic, stores and launches. Short tile names are intentional: the full tensors are `query`, `key` and `value` in Python.

Notation, causality, and padding

Vectors versus rows

In equations, $\boldsymbol{q}_t, \boldsymbol{k}_t \in \mathbb{R}^{D_k}$ and $\boldsymbol{v}_t, \boldsymbol{o}_t \in \mathbb{R}^{D_v}$ are column vectors. A state is $\mathbf{S}_t \in \mathbb{R}^{D_k \times D_v}$, so its read is $\mathbf{S}_t^\top \boldsymbol{q}_t$. A chunk stacks vectors as rows: $\mathbf{Q}, \mathbf{K} \in \mathbb{R}^{C \times D_k}$ and $\mathbf{V} \in \mathbb{R}^{C \times D_v}$. Thus the same read is $\mathbf{Q}\mathbf{S}_{\text{in}}$.

Global tokens are numbered $1, \ldots, T$. Chunk c starts after token cC, with local positions $r = 1, \ldots, C$. Its entry state is $\mathbf{S}^{(c)} = \mathbf{S}_{cC}$; its exit state is $\mathbf{S}^{(c+1)}$. Python indices start at zero. Translate indices carefully when creating masks.

Define the inclusive and strict masks by

$$M_{ru} = \mathbb{I}[u \le r], \qquad M^-_{ru} = \mathbb{I}[u < r].$$

The output reads the state *after* the current token's update. Therefore output scores include the diagonal. The delta correction depends only on earlier corrections, so its triangular system excludes the diagonal.

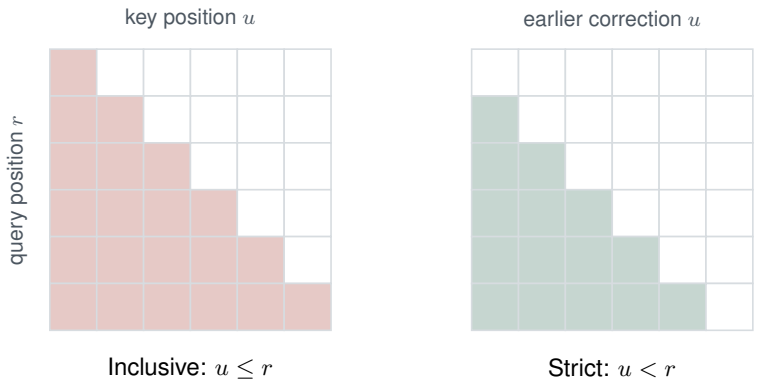


Figure 1. Two masks with different jobs. Warm red entries contribute to outputs; green entries encode dependence on earlier delta corrections. Light-gray cell boundaries preserve the matrix grid.

Padding preserves the state

Launchers pad sequence length to $\widetilde{T} = C\lceil T/C \rceil$. Padded queries, keys, values, and write strengths are zero; padded log gates are zero, giving decay one. These tokens cannot change the state. Trim the output to T; the state after padded execution remains the state after real token T.

Keys in delta-family problems are supplied with unit L_2 norm. Gate generation, projections, feature transformations, short convolutions, and output normalization are outside these kernel exercises. The linear attention defined here is unnormalized: the operator has no softmax or additional denominator.

Mathematical notation and translation to code

Notation conventions

Scalars are italic, column vectors bold italic, and matrices bold upright. Tensor token vectors are stored as rows. A superscript $\top$ denotes transpose, $\odot$ elementwise multiplication, and $\mathrm{Diag}(\boldsymbol{a})$ a diagonal matrix in equations. Exponentials are elementwise. Query scaling is omitted from all mathematical formulas. The code retains its optional `scale` argument; use `scale=1.0` when reproducing the equations and worked examples.

Table 2. Notation conventions.

Symbol	Space or range	Interpretation
t, u	$\{1, \dots, T\}$	Global token positions
c	$\{0, \dots, N-1\}$	Chunk index, $N = \lceil T/C \rceil$
r, j	$\{1, \dots, C\}$	Local positions within a chunk
d, a	Key/value coordinates	$d \le D_k$, $a \le D_v$
$\boldsymbol{q}_t, \boldsymbol{k}_t$	$\mathbb{R}^{D_k}$	Query and key columns
$\boldsymbol{v}_t, \boldsymbol{u}_t$	$\mathbb{R}^{D_v}$	Value and actual correction columns
$\mathbf{S}_t$	$\mathbb{R}^{D_k \times D_v}$	Memory after token t
$\mathbf{S}^{(c)}$	$\mathbb{R}^{D_k \times D_v}$	Entry memory of chunk c, equal to $\mathbf{S}_{cC}$
$\boldsymbol{g}_r, \boldsymbol{b}_r$	$\mathbb{R}^{D_k}$	Channel log gate and chunk-local inclusive prefix
g_r, b_r	$\mathbb{R}$	Scalar log gate and chunk-local prefix
$\mathbf{A}$	$\mathbb{R}^{C \times C}$	Inverse of a unit lower-triangular system
$\mathbf{W}, \mathbf{U}, \mathbf{P}$	$C \times D_k$, $C \times D_v$, $C \times D_v$	Entry-state factor, base pseudo-values, actual corrections

Batch and head indices are suppressed in derivations: every equation applies independently to one batch/head. The symbol $\mathbf{P}$ always means a matrix of actual corrections; it is not an attention probability matrix. Likewise, $\mathbf{A}$ is a triangular inverse, not a softmax attention matrix.

The row/column translation rule

If $\mathbf{K}$ stacks $\boldsymbol{k}_r^\top$ as rows, then

$$(\mathbf{K}\mathbf{S}_{\text{in}})_{r,:} = \boldsymbol{k}_r^\top \mathbf{S}_{\text{in}}, \qquad (\mathbf{K}^\top \mathbf{P})_{d,a} = \sum_{r=1}^{C} K_{rd} P_{ra}.$$

The first predicts values from memory; the second writes corrections into memory. Neither requires $D_k = D_v$. Draw the contracted axis above a matrix product before translating it into `tl.dot`.

One-based mathematics, zero-based code

Mathematical local token r is code offset `r - 1`. An inclusive mask $j \le r$ becomes `row_offsets[:, None] >= row_offsets[None, :]`; a strict mask $j < r$ uses `>`. Mathematical chunk entry $\mathbf{S}^{(c)}$ is `states[:, :, c]`. After processing that chunk, store its exit in `states[:, :, c + 1]`.

Elementwise decay has a specified axis

For a channel vector $\boldsymbol{a} \in \mathbb{R}^{D_k}$,

$$[\mathrm{Diag}(\boldsymbol{a})\mathbf{S}]_{d,a'} = a_d S_{d,a'}.$$

In code, the gate vector broadcasts as `decay[:, None]` over a state tile. A scalar gate broadcasts over both axes. This distinction explains why KDA needs channel-aware scores and why Gated DeltaNet can apply relative decay after an ordinary dot product.

B Triton shape and memory reference

Core operations

Table 3. Core operations.

Operation	Role in this assignment
`tl.program_id(axis)`	Select chunk, key tile, value tile, or batch/head
`tl.arange(0, BLOCK)`	Create offsets for a fixed-size tile
`tl.load / tl.store`	Move tile data with the documented pointers
`tl.sum(x, axis=0)`	Reduce key channels in a recurrent read
`tl.cumsum(x, axis=0)`	Inclusive token-prefix scan within a chunk
`tl.dot(a, b)`	Matrix products for state, WY, and output kernels
`tl.trans(x)`	Transpose a loaded matrix tile
`tl.where(mask, x, 0)`	Zero invalid linear weights
`tl.exp(x)`	Natural exponential of an FP32 log gate

Matrix products and their shapes

Precision boundaries

Keep recurrent state arithmetic in FP32. For chunked tensor-core products, load BF16 operands and accumulate in FP32, using casts where the pinned Triton version requires compatible operand types. Keep triangular-system calculations, gate prefixes, and direct diagonal gated scores in FP32. Casting the carried state to a matrix-product operand dtype introduces rounding; the recurrence reference measures the resulting error.

The assignment uses natural-log gates and `tl.exp`. If you choose base-two exponentials, transform the exponent with $\log_2(e)$ consistently. Query scaling and gate-log conversion are separate operations.

Table 4. Matrix products and their shapes.

Product	Result	Purpose
$\mathbf{K}^\top\mathbf{V}$	$D_k \times D_v$	Additive state write
$\mathbf{QS}_{\text{in}}$	$C \times D_v$	Read earlier chunks
$\mathbf{QK}^\top$	$C \times C$	Local token interaction
$\mathbf{AD}_\beta\mathbf{K}$	$C \times D_k$	Delta entry-state factor
$\mathbf{AD}_\beta\mathbf{V}$	$C \times D_v$	Base pseudo-values
$\mathbf{WS}_{\text{in}}$	$C \times D_v$	Subtract entry-state predictions
$\mathbf{K}_{\text{end}}^\top\mathbf{P}$	$D_k \times D_v$	Decayed corrected state write

Table 5. Debugging by invariants.

Symptom	First invariant to inspect
First token wrong	Current-token inclusion; initial-state read; scale
First chunk correct, later chunks wrong	Entry/exit state timing; local prefix reset
Only $D_k = 128$ fails	Sum all key tiles before finalizing pseudo-values
Only $D_v = 128$ fails	Launch/store both value tiles
Delta-family error	Strict WY mask; row-wise beta; prediction before write
Strong gates produce NaN	Masked exponent arguments; growing e^{-b} factors
Segmented calls disagree	Final state; padding neutrality; scan restart
Output right, final state wrong	Exit-decayed keys; state scaling; final store
Scale changes memory state	Query scale incorrectly entered WY/state path
Nondeterministic scores	Uninitialized upper-triangle buffer entries

Debugging by invariants

A recommended debugging order

1. Run a one-token example with nonzero initial state.
2. Inspect the state and output separately in a toy two-channel case.
3. Test a single complete chunk before multiple chunks.
4. Check gate prefixes and score matrices entry by entry.
5. Check the triangular solve and WY factors before pseudo-values.
6. Check pseudo-values before chunk exit states.
7. Test partial chunks and sequence splits.
8. Run BF16 stress tests, then measure timing and memory.

What code is supplied versus hidden

The starter pack supplies signatures, allocations, zero initialization, padding, launch grids/order, pointer arithmetic, and most loads/stores. Named TODO regions correspond to the mathematical steps in this handout. Incomplete PyTorch functions should fail with a descriptive `NotImplementedError`; incomplete Triton regions should fail clearly during compilation. They must not return uninitialized output buffers.

The optimized gated-score primitive is introduced in Problem 8. Before that point, Problem 4 uses the direct baseline. Each student mathematical primitive is written once and reused; later problems compose previous work.

C Starter-file map and reusable primitives

Buffer names and mathematical symbols

`gate_cumsum` stores log prefixes b; `kernel_matrix` stores strict, unscaled key–key scores; `scores` stores inclusive query–key scores (already scaled in P4/P9). `inverse`, `w`, `u` and `pseudo_values` correspond to $\mathbf{A}$, $\mathbf{W}$, $\mathbf{U}$ and $\mathbf{P}$. `kernel_args` holds launch constants; `grid` specifies ownership. The kernel-interface section of this reference explains dimension names and pointer/tile naming.

Exact named regions in the current code pack

The main problem listings illustrate the mathematical steps. The tables below map them to the exact region identifiers in the actual Python files. To obtain an identifier, concatenate its prefix, a period, and one region suffix. For example, prefix `P6.WY` and suffix `W` identify `P6.WY.W`. The supplied loops, loads, stores and launchers remain outside those regions.

Table 6. Exact named regions in the current code pack.

File	Region prefix	Region suffixes
problem_1.py	P1.RECURRENT	UPDATE, READ
	P1.CHUNK	OUTPUT, STATE
problem_2.py	P2.RECURRENT	UPDATE, READ
problem_3.py	P3.STATE	UPDATE
	P3.OUTPUT	PRODUCTS, MASK, COMBINE
problem_4.py	P4.RECURRENT	UPDATE, READ
	P4.SCAN	PREFIX
	P4.STATE	EXIT_KEYS, UPDATE
	P4.OUTPUT	LOCAL, ENTRY

Supplied infrastructure

`common.py` validates shapes, pads token axes, prepares contiguous buffers, provides pointer helpers and formats returns. It performs no attention update. `reference.py` supplies slow independent FP64 recurrence definitions to the checker, not student implementations. `provided_gated_scores.py` is the specified temporary P4 baseline. `grader_kernels.py` launches your P6 inverse for an isolated check; it contains no inverse solution.

Starter-file map: continued

Table 7. Starter-file map.

File	Region prefix	Region suffixes
problem_5.py	P5.RECURRENT	RESIDUAL, WRITE, READ
problem_6.py	P6.INVERSE	SUBSTITUTE, MERGE
	P6.WY	GRAM, STRICT, W, U
	P6.STATE	PSEUDO, UPDATE
problem_7.py	P7.RECURRENT	DECAY, RESIDUAL, WRITE, READ
	P7.WY	GATES, GRAM, STRICT, W, U
	P7.STATE	PREFIX, PSEUDO, UPDATE
	P7.OUTPUT	GATES, PRODUCTS, COMBINE

What P6 reuses and what P7 extends

P6 directly imports `_chunk_output_forward` from P3 and supplies pseudo-values in the value argument. Its output mathematics is the same linear read of corrected writes. P7 directly imports P6's `triangular_inverse`; its recurrent, WY, state and output regions extend earlier structures with scalar forgetting.

Starter-file map: continued

Table 8. Starter-file map.

File	Region prefix	Region suffixes
`problem_8.py`	`P8.RECURRENT`	`DECAY`, `RESIDUAL`, `WRITE`, `READ`
	`P8.DIAGONAL`	`SCORES`
	`P8.SCORES`	`ANCHOR`, `STRICT`, `INCLUSIVE`
`problem_9.py`	`P9.WY`	`LOWER`, `W`, `U`
	`P9.STATE`	`REDUCE`, `UPDATE`
	`P9.OUTPUT`	`LOCAL`, `ENTRY`

The final KDA dependencies

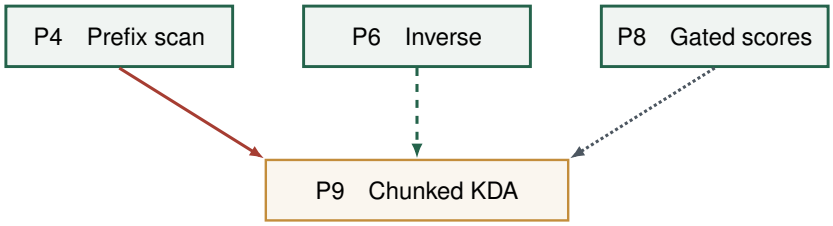


Figure 2. Executable reuse of earlier student primitives. Labels identify each dependency, while line styles distinguish the three paths without requiring color.

P4 can additionally call P8's score builder through its `score_builder` argument. This optional integration replaces P4's temporary PyTorch score baseline with your stable Triton computation. The P9 path uses P8 directly.

Checker cookbook: interpret tests and numerical error

Which command answers which question?

What a failed check tells you

An explicit missing-subpart exception means the named student region is still incomplete. A nonfinite-output failure suggests overflow or an uninitialized buffer. A state-checkpoint failure points to the boundary update even if the output kernel is correct. A standalone inverse-residual failure should be fixed before interpreting a full WY failure. Input-mutation failures mean a caller-owned tensor changed; use the supplied copied state rather than writing into the input state directly.

The current acceptance rule

Let a and r be provisional absolute and relative tolerances for the input dtype. The public checker enforces both

$$E_{\max} \le a + r \max_i |Y_i|, \qquad \mathrm{RMS}(\widehat{\mathbf{Y}} - \mathbf{Y}) \le a + r\,\mathrm{RMS}(\mathbf{Y}).$$

It also reports relative RMS error. The absolute term matters when the reference is nearly zero, so a large reported relative ratio alone does not determine pass/fail. Current tolerances are FP32

Table 9. Which command answers which question?.

Command	Interpretation
`-check-templates`	Can the files be parsed, and do their APIs/imports follow the release structure? No numerical execution.
`-p1 -device cpu`	Do both PyTorch definitions match the independent recurrence, including arbitrary toy dimensions and chunk sizes?
`-p6`	Do the inverse, WY intermediates, state/output kernels and complete DeltaNet path work?
`-p8`	Do stable scores satisfy both masks and integrate into GLA, and does recurrent KDA match the reference?
`-p9`	Does the final composition handle chunk boundaries, gates, state passing and independent key/value dimensions?
`-all -dtype bfloat16`	Do all selected operators satisfy the lower-precision checks?
`-p9 -benchmark`	Does P9 pass correctness, and then what is its full-wrapper forward time?

$(a, r) = (0.005, 0.01)$, FP16 $(0.01, 0.02)$, BF16 $(0.03, 0.05)$, and FP64 $(10^{-9}, 10^{-9})$. These values are provisional; course GPU validation must establish official limits before graded release. They are not permission to change the mathematical operator.

Correctness and performance are different evidence

The checker compares outputs and final states to independent recurrences, but output equality cannot prove that a student used a chunked Triton path. Source review checks the required implementation and reuse. Timing excludes compilation and warmup and currently includes validation and allocations in the wrapper; report GPU, dtype and toolchain alongside time. No hard speed threshold or measured performance curve is specified in this handout.

Companion reading: attention, memory, and delta corrections

Yang's talk [3] is a companion to P1–P7. The page numbers below refer to physical PDF pages, counting the title page as page 1. Read the assignment's equations alongside the slides: the slides store value-by-key memory, the transpose of our key-by-value memory. The talk predates KDA; P8–P9 use the Kimi Linear paper instead [8]. The supplied PDFs contain the definitions needed for

Table 10. Companion reading: attention, memory, and delta corrections.

Problem	Suggested reading and question to carry into the kernels
P1	Talk pp. 3–7; normalized linear attention [2]. Which operation prevents reassociating softmax?
P2	Talk pp. 5–7. What does a state column remember?
P3	Talk pp. 8–10. How do prefix memory and local causal products partition the history?
P4	Talk pp. 24–26; GLA [5]. Which gates act on a write made at token j?
P5	Talk pp. 12–16 and 35. Why correct the prediction rather than add the entire value?
P6	Talk pp. 18–21 and the chunkwise derivation [4]. Why is the correction system triangular?
P7	Talk pp. 24–27; Gated DeltaNet [7]. What can a scalar gate forget that a key-directed correction cannot?
P8	Kimi Linear, Section 3.1 [8]. Why does channel decay stay inside the dot product?
P9	Kimi Linear, Section 3.1 [8]. How do local factors define the full affine chunk map?

the exercises; these external readings supply additional perspectives rather than build or runtime dependencies. Each problem opens with mathematical background and a *Before coding* question, then develops the exact kernel equations.

D Recurrence summary

All outputs use $\boldsymbol{o}_t = \mathbf{S}_t^\top \boldsymbol{q}_t$. The table specifies the state update in chronological order.

Table 11. Recurrence summary.

Mechanism	State update
Linear	$\mathbf{S}_t = \mathbf{S}_{t-1} + \boldsymbol{k}_t \boldsymbol{v}_t^\top$
GLA	$\mathbf{S}_t = \mathrm{Diag}(e^{\boldsymbol{g}_t})\mathbf{S}_{t-1} + \boldsymbol{k}_t \boldsymbol{v}_t^\top$
DeltaNet	$\boldsymbol{u}_t = \beta_t(\boldsymbol{v}_t - \mathbf{S}_{t-1}^\top \boldsymbol{k}_t)$; $\mathbf{S}_t = \mathbf{S}_{t-1} + \boldsymbol{k}_t \boldsymbol{u}_t^\top$
Gated DeltaNet	$\overline{\mathbf{S}}_t = e^{g_t}\mathbf{S}_{t-1}$; $\boldsymbol{u}_t = \beta_t(\boldsymbol{v}_t - \overline{\mathbf{S}}_t^\top \boldsymbol{k}_t)$; $\mathbf{S}_t = \overline{\mathbf{S}}_t + \boldsymbol{k}_t \boldsymbol{u}_t^\top$
KDA	$\overline{\mathbf{S}}_t = \mathrm{Diag}(e^{\boldsymbol{g}_t})\mathbf{S}_{t-1}$; $\boldsymbol{u}_t = \beta_t(\boldsymbol{v}_t - \overline{\mathbf{S}}_t^\top \boldsymbol{k}_t)$; $\mathbf{S}_t = \overline{\mathbf{S}}_t + \boldsymbol{k}_t \boldsymbol{u}_t^\top$

The unifying chunk template

For delta-family methods, solve a strictly lower system to obtain $\mathbf{W}, \mathbf{U}$. Form $\mathbf{P} = \mathbf{U} - \mathbf{W}\mathbf{S}_{\mathrm{in}}$. Advance the state using keys transported to the chunk end, and compute output using queries transported from the chunk start plus local inclusive interactions. For plain linear/GLA methods, values already play the role of local writes, so the delta system is unnecessary.

Sources and scope

The derivations follow the papers below. All required starter code, recurrence references, grader helpers, and documentation sources are included in this assignment folder. The handout adopts a uniform head-first layout, natural-log gates, and one output-scaling convention. Upstream APIs may use other layouts or gate-generation options; compare equations after translating those conventions.

References

[1] A. Vaswani et al. *Attention Is All You Need*. 2017.
https://arxiv.org/abs/1706.03762

[2] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret. *Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention*. 2020.
https://arxiv.org/abs/2006.16236

[3] S. Yang. *Toward More Expressive yet Scalable RNNs: DeltaNet and Its Variants*. Talk, July 15, 2025.
https://sustcsonglin.github.io/assets/pdf/deltanet_talk_slides.pdf

[4] S. Yang. *DeltaNet Explained (Part II)*. 2024.
https://sustcsonglin.github.io/blog/2024/deltanet-2/

[5] S. Yang et al. *Gated Linear Attention Transformers with Hardware-Efficient Training*. 2023/2024.
https://arxiv.org/abs/2312.06635

[6] S. Yang et al. *Parallelizing Linear Transformers with the Delta Rule over Sequence Length*. 2024.
https://arxiv.org/abs/2406.06484

[7] S. Yang, J. Kautz, and A. Hatamizadeh. *Gated Delta Networks: Improving Mamba2 with Delta Rule*. 2024/2025.
https://arxiv.org/abs/2412.06464

[8] Kimi Team. *Kimi Linear: An Expressive, Efficient Attention Architecture*. 2025.
https://arxiv.org/abs/2510.26692

[9] Qwen Team. *Qwen3-Next: Towards Ultimate Training & Inference Efficiency*. 2025.
https://qwen.ai/blog?id=qwen3-next

[10] Moonshot AI. *Kimi K3: Model Introduction and Architecture*. 2026.
https://github.com/MoonshotAI/Kimi-K3

[11] FLA contributors. *Flash Linear Attention*. Reference implementations.
https://github.com/fla-org/flash-linear-attention